

ENTREGA 1 - MODELAGEM E ARQUITETURA

ENTREGA 1 - MODELAGEM E ARQUITETURA

Disciplina: Desenvolvimento Web

Equipe: Marcos Eduardo, Guilherme Araujo, Lauro Vitor, Pedro Henrique

Professor(a):

Prof. Felipe Pires Ferreira

Data: 28/04/2026

Sistema Space Games

Sistema web para catálogo de jogos, biblioteca, favoritos e instalados.

Projeto: Sistema de Gestão Comercial / Loja de Jogos

Repositório GitHub: <https://github.com/Guilherme954/SpaceGames>

Base analisada: repositório GitHub + pasta spacegames_backend_working

Data: 2026

1. Descrição do sistema

O Space Games é um sistema web voltado para uma loja/plataforma de jogos digitais. O projeto apresenta uma interface para navegação por jogos, busca, filtros de biblioteca, jogos instalados e favoritos. A pasta de backend enviada contém uma API em Django REST Framework, com autenticação JWT, cadastro de usuários e controle da biblioteca individual de cada usuário.

No contexto da disciplina, o sistema se encaixa como uma solução de gestão comercial aplicada a uma loja de games. Na Entrega 1, o foco está na modelagem, arquitetura inicial, versionamento e cadastro básico de usuários/clientes e produtos/jogos. As funcionalidades de vendas, itens de venda e relatórios podem ser evoluídas para a Entrega 2.

2. Requisitos funcionais

Código	Descrição
RF01	Permitir cadastro de usuário/cliente por meio do endpoint de registro.
RF02	Permitir login com geração de token JWT.
RF03	Listar jogos/produtos disponíveis no catálogo.
RF04	Permitir busca de jogos por título, descrição ou gênero.
RF05	Permitir filtrar jogos por favoritos e instalados.
RF06	Permitir que usuário autenticado visualize sua biblioteca.
RF07	Permitir marcar ou desmarcar jogo como favorito.
RF08	Permitir marcar ou desmarcar jogo como instalado.
RF09	Permitir cadastro e manutenção dos jogos/produtos pelo painel administrativo do Django.
RF10	Preparar a modelagem para evolução futura de vendas, itens de venda e relatórios.

3. Requisitos não funcionais

Código	Descrição
RNF01	Segurança: uso de autenticação JWT e senhas armazenadas de forma criptografada pelo Django.
RNF02	Manutenibilidade: separação do sistema em apps Django, serializers, views e models.
RNF03	Escalabilidade: API REST paginada, permitindo crescimento do catálogo.
RNF04	Usabilidade: interface visual simples, escura, com cards, busca e filtros.
RNF05	Portabilidade: backend executável localmente com Python e banco SQLite.
RNF06	Versionamento: código versionado em repositório GitHub público.
RNF07	Integração: front-end preparado para consumir endpoints REST do backend.

4. Arquitetura proposta

A arquitetura adotada é em camadas, separando interface, API, regra de negócio e persistência. Essa organização facilita manutenção, testes e evolução do sistema.

Camada	Tecnologia / responsabilidade
Interface Web	HTML, CSS e JavaScript. Exibe catálogo de jogos, busca, filtros, favoritos e instalados.
API REST	Django REST Framework. Expõe endpoints em JSON para autenticação, jogos e biblioteca.
Autenticação	SimpleJWT. Gera access token e refresh token para rotas protegidas.

Regra de negócio	Models, serializers e views do Django. Controla jogos, biblioteca, favoritos e instalados.
Persistência	SQLite no backend atual. Pode ser adaptado para MySQL na evolução do projeto.
Versionamento	GitHub: https://github.com/Guilherme954/SpaceGames

5. Padrões de projeto escolhidos

Padrão	Aplicação no projeto
Arquitetura em Camadas	Separação entre front-end, API, regra de negócio e banco de dados.
MVT/MVC do Django	Models representam dados, views controlam requisições e templates/front-end representam a interface.
REST	Endpoints organizados por recursos, retornando dados em JSON.
Serializer / DTO	Serializers convertem models para JSON e validam dados de entrada/saída.
ORM / Repository	Django ORM abstrai o acesso ao banco de dados.
Token Based Authentication	JWT protege rotas e identifica o usuário autenticado.

6. Endpoints da API identificados no backend

Método	Endpoint	Finalidade
POST	/api/auth/register/	Cadastrar usuário/cliente.
POST	/api/auth/token/	Realizar login e gerar token JWT.
POST	/api/auth/token/refresh/	Renovar token JWT.
GET	/api/games/	Listar jogos/produtos ativos, com busca e filtros.
GET	/api/library/	Listar biblioteca do usuário autenticado.
POST	/api/games/<id>/toggle-favorite/	Marcar ou desmarcar jogo como favorito.
POST	/api/games/<id>/toggle-installed/	Marcar ou desmarcar jogo como instalado.

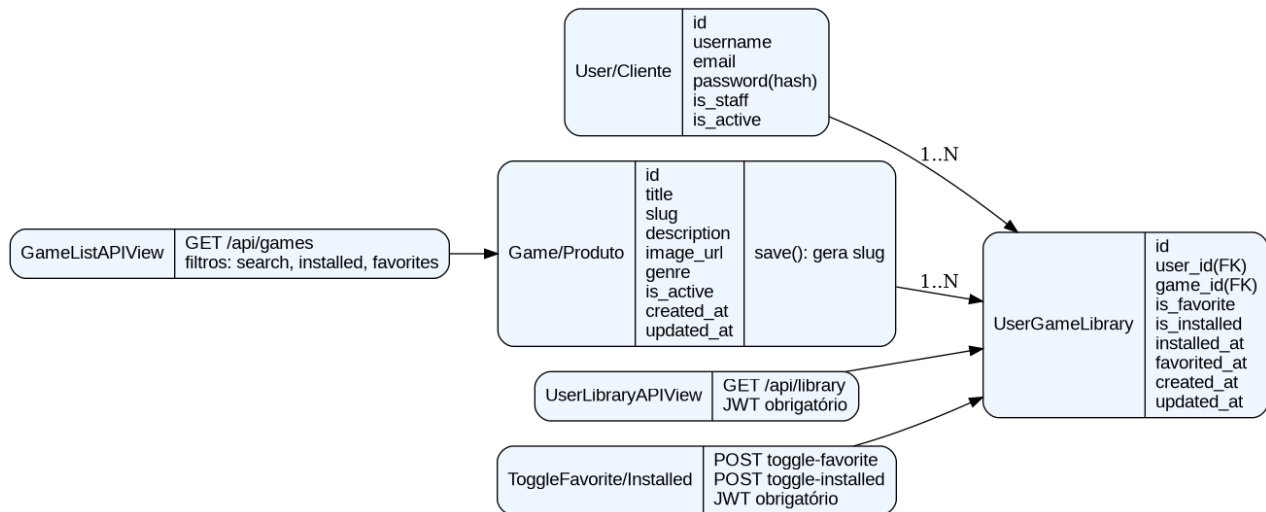
7. Cadastro de clientes e produtos

No projeto atual, o cadastro de clientes é representado pelo cadastro de usuários, feito pelo endpoint /api/auth/register/. O Django armazena o usuário na tabela auth_user, com senha criptografada. Para a Entrega 2, o cadastro pode ser ampliado com campos de cliente, como CPF, telefone e endereço.

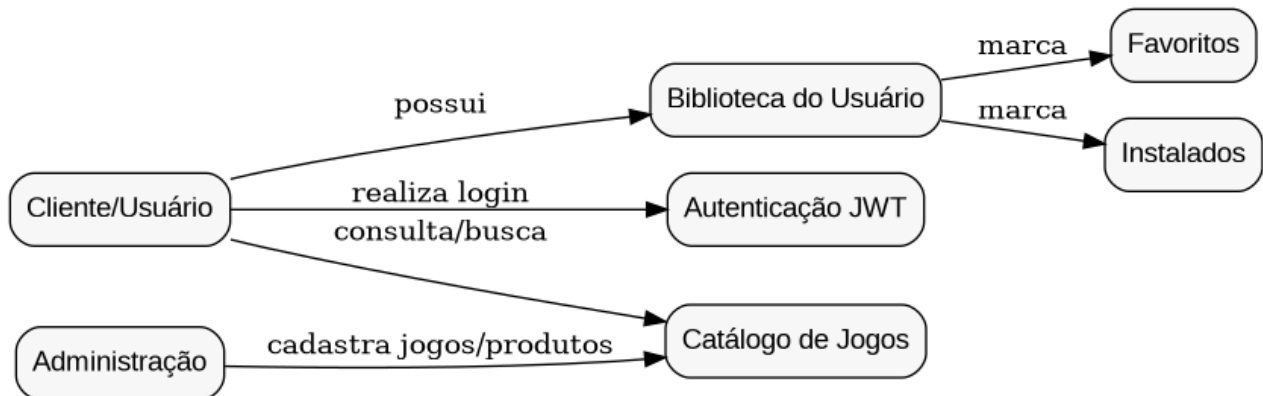
O cadastro de produtos é representado pelo cadastro de jogos, usando o model Game. Cada jogo possui título, slug único, descrição, imagem, gênero e status ativo. A manutenção dos jogos/produtos pode ser feita pelo painel administrativo do Django ou por endpoints administrativos a serem adicionados.

Entidade exigida	Como aparece no projeto atual	Observação
Cliente	User / auth_user	Cadastro e autenticação de usuários. Pode receber extensão com CPF, telefone e endereço.
Produto	Game / games_game	Representa os jogos/produtos exibidos no catálogo.
Venda	Não implementada no backend atual	Recomendado implementar na Entrega 2.
Itens de venda	Não implementado no backend atual	Recomendado implementar na Entrega 2.
Relatórios	Não implementado no backend atual	Recomendado implementar na Entrega 2.

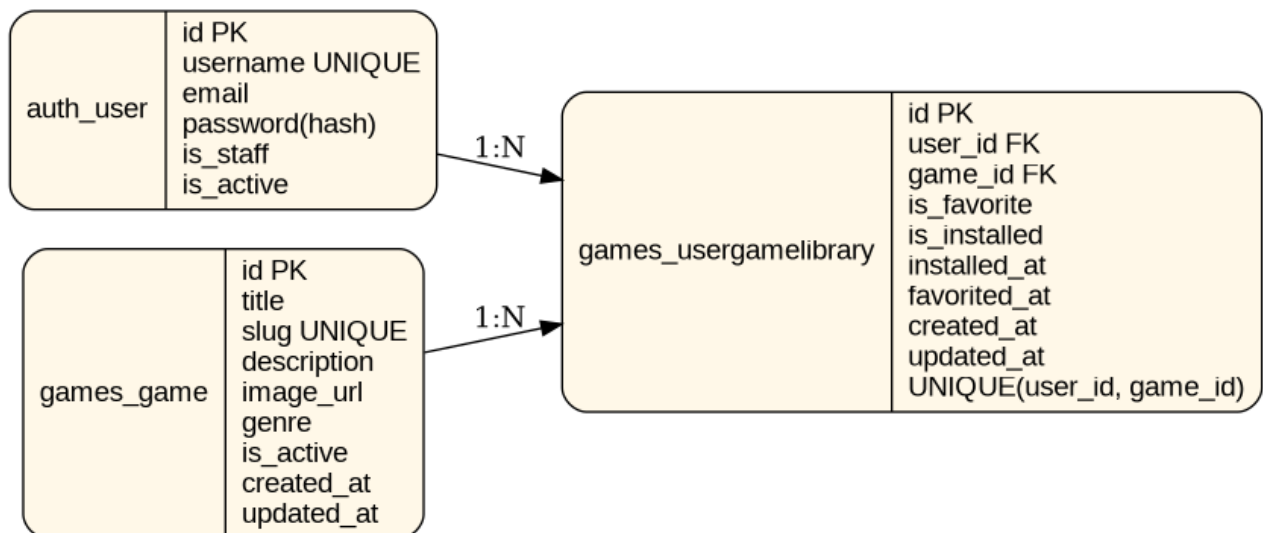
8. Diagrama de Classe



9. Diagrama de Domínio



10. Diagrama lógico do banco



11. Tabelas principais do banco

Tabela	Campos principais	Finalidade
auth_user	id, username, email, password, is_staff, is_active	Armazena usuários/clientes e administradores.
games_game	id, title, slug, description, image_url, genre, is_active	Armazena jogos/produtos do catálogo.
games_usergamelibrary	id, user_id, game_id, is_favorite, is_installed	Relaciona usuários aos jogos da biblioteca.

12. Script de criação do banco de dados

O arquivo SQL atualizado foi gerado separadamente com o nome: script_banco_spacegames_atualizado.sql. Ele representa a modelagem lógica principal do backend atual, usando SQLite, com chaves primárias, chaves estrangeiras, índice e restrição UNIQUE para impedir duplicidade de jogo por usuário na biblioteca.

13. Instruções para executar o sistema

1. Baixar ou clonar o repositório: `git clone https://github.com/Guilherme954/SpaceGames`
2. Entrar na pasta do backend: `cd spacegames_backend`
3. Criar ambiente virtual: `python -m venv venv`
4. Ativar o ambiente virtual: `venv\Scripts\activate`
5. Instalar dependências: `pip install -r requirements.txt`
6. Executar migrações: `python manage.py migrate`
7. Popular jogos iniciais: `python manage.py seed_games`
8. Iniciar servidor: `python manage.py runserver`
9. Abrir o front-end pelo arquivo `index.html` ou configurar o consumo da API pelo JavaScript.

14. Link do repositório GitHub

<https://github.com/Guilherme954/SpaceGames>

Observação: no momento da análise, o repositório público contém os arquivos `index.html`, `script.js` e `style.css`. A pasta enviada separadamente contém o backend Django. Para a entrega ficar completa no GitHub, recomenda-se subir também a pasta do backend, a pasta database com o script SQL e uma pasta docs com este documento em PDF.

15. Checklist da Entrega 1

Item exigido	Situação neste material
Documento PDF com descrição do sistema	Incluído neste documento; exportar para PDF.
Requisitos funcionais	Incluído.
Requisitos não funcionais	Incluído.
Arquitetura proposta	Incluído.
Padrões de projeto escolhidos	Incluído.
Diagrama de Classe	Incluído.
Diagrama de Domínio	Incluído.
Diagrama lógico do banco	Incluído.
Cadastro de Clientes e Produtos	Descrito conforme implementação atual: User/Cliente e Game/Produto.
Script de Criação do Banco	Gerado em arquivo separado.
Repositório GitHub criado	Incluído link do repositório.